

Date of current version April 21, 2026.

Digital Object Identifier to be assigned

Gap-Aware Candidate Reranking for Diffusion-Guided Multi-Strategy Floorplanning

SHASHANK¹, (Member, IEEE)

¹Independent Researcher, Austin, TX 78703 USA (e-mail: sshashan@alumni.usc.edu)

Corresponding author: Shashank (e-mail: sshashan@alumni.usc.edu).

ABSTRACT Multi-strategy VLSI floorplanners combine heterogeneous candidate generators (shelf packing, sequence-pair simulated annealing, analytical placement, and learned neural priors) and select among them using an internal ranking cost. In such pipelines, the choice of that ranking cost is an important but under-analysed design decision. We frame this paper as a benchmark-specific study of metric-faithful candidate selection on FloorSet. We show that a commonly used raw-magnitude ranking cost is systematically mis-calibrated with respect to the benchmark's gap-based contest metric, causing the solver to discard low-wirelength candidates for reasons unrelated to the final score. We therefore introduce a gap-aware reranker that operates on percentile-normalised half-perimeter wirelength and area statistics, together with a constraint-aware violation proxy that mirrors the contest's exact soft-violation denominator. Integrated into a diffusion-guided multi-strategy solver, this reranker improves the composite contest score on the full 100-case validation set from 1.43 to 1.40228 (1.94% relative reduction). As supporting engineering adaptations, we report two DiT-guided candidate-path variants that broaden the candidate pool, and a tiered stochastic sampling schedule that reduces end-to-end evaluation wall-time by 39% with no score loss. A cross-benchmark evaluation on the polygonal FloorSet-Prime test set, using the same solver with no hyperparameter re-tuning, yields $S_{100} = 1.54862$ with 91/100 instances feasible, suggesting that the rerank and runtime-design choices remain effective on a distinct problem class. Finally, we characterise the residual error as an empirical two-regime candidate structure, in which learned priors produce low-wirelength / higher-violation candidates while classical packers produce higher-wirelength / lower-violation candidates, and present controlled negative results for a scaled-up DiT variant and an unbounded analytical-placement path that help explain this remaining performance bottleneck.

INDEX TERMS Candidate selection, metric calibration, multi-strategy floorplanning, diffusion transformer, machine learning for electronic design automation, simulated annealing, FloorSet benchmark.

I. INTRODUCTION

Block-level floorplanning, which assigns positions and shapes to rectangular logic blocks on a fixed die area, is a canonical NP-hard problem whose quality directly sets downstream routing, timing, and manufacturing margins in modern VLSI design [1], [2]. Classical solvers rely on combinatorial search over sequence-pair or B^* -tree representations [3], [4], analytical placement with smooth wirelength models [9], [10], or greedy skyline-style packers [5]. The last five years have added learned placement priors (graph networks, reinforcement learning agents, and more recently diffusion models) that propose candidate layouts which downstream legalisation then

repairs [14]–[18]. Production-grade pipelines increasingly combine these families, because no single strategy dominates on every instance.

The FloorSet benchmark [21], adopted as the ICCAD 2026 CAD Contest Problem C, makes this regime quantitative. It evaluates floorplanning under a composite cost that jointly penalises half-perimeter wirelength (HPWL), bounding-box area, five categories of soft constraint violations, and runtime:

$$C = \left(1 + \frac{1}{2}(\Delta_{\text{HPWL}} + \Delta_{\text{area}})\right) e^{2V_{\text{rel}}} \max(0.7, R^{0.3}). \quad (1)$$

Here Δ_{HPWL} and Δ_{area} are gaps relative to ground-truth labels, $V_{\text{rel}} \in [0, 1]$ is a normalised soft-violation count, and R

is per-case runtime normalised by the submission's own median. An infeasible solution is assigned $C = 10$, and the overall score is an exponentially-weighted mean across all 100 validation instances. The exponential dependence on V_{rel} makes constraint satisfaction disproportionately important: a 0.1 increase in V_{rel} multiplies cost by $e^{0.2} \approx 1.22$, which is comparable to a 44% HPWL-gap regression.

In such a metric-sensitive regime, a multi-strategy solver's candidate ranking function, that is, the internal cost used to choose one layout among many produced by different generators, is not an implementation detail. It is a policy that silently encodes how HPWL, area, and violations trade off at the margin. If this policy is not aligned with the evaluation metric, the solver discards good candidates for reasons unrelated to the downstream score.

Accordingly, we present this paper as an applied, benchmark-specific study of metric-faithful candidate selection inside a multi-strategy solver. The gap-aware reranker is the paper's single primary methodological contribution; the DiT-path and runtime changes are supporting solver adaptations whose value lies in candidate-pool coverage and efficiency rather than an additional S_{100} gain.

A. CONTRIBUTIONS

This paper makes the following contributions.

- 1) **A calibrated reranking formulation, with analysis, as the paper's primary method contribution.** We show that the raw-magnitude internal cost commonly used in such pipelines is mis-calibrated relative to FloorSet's gap-based contest metric: on typical instances the absolute bounding-box area is 5–10 $\times$ larger than the absolute HPWL, so linear combinations of raw magnitudes systematically distort the intended trade-off. We then introduce a gap-aware reranker that normalises HPWL and area by a fixed per-case percentile ($p = 0.3$) and evaluates violations using the contest's exact soft-violation denominator. Integrated into our multi-strategy solver, this reranker improves the full 100-case FloorSet-Lite score from 1.43 to **1.40228** with every instance feasible (§V, §VII, Proposition 1).
- 2) **Supporting solver adaptations that change operating envelope rather than aggregate score.** We introduce two DiT-guided candidate-path adaptations (DiT-abacus and DiT-cluster) that broaden the candidate pool and improve feasibility profile on difficult instances, and a tiered stochastic-sampling schedule that reduces end-to-end evaluation wall time by 39% (6728 $\rightarrow$ 4112 seconds) with no score regression. We explicitly do not claim these supporting modules further improve S_{100} beyond the reranker; their value is candidate-pool coverage, solver integration, and runtime reduction (§VI, §VII-C).
- 3) **Empirical analysis of residual limitations and transfer behaviour.** We document a short-subset winner breakdown, a case-70 post-mortem, two controlled negative interventions (a 2 $\times$ -parameter DiT variant trained with an auxiliary HPWL loss, and an unbounded analytical-

placement path with robust legalisation fallback) that regress the score by 14% and 28%, and a cross-benchmark evaluation on FloorSet-Prime where the same solver reaches $S_{100} = 1.54862$ with 91/100 instances feasible. Together these results expose an empirical two-regime candidate structure that explains both the remaining Lite gap and the graceful degradation on Prime (§VIII, §VII-F, §IX).

Section II reviews related work in learned placement, multi-strategy solvers, and candidate-selection policies. Section III formalises the contest metric and constraint taxonomy. Section IV describes the baseline multi-strategy architecture. Section V presents the reranking diagnosis and formulation, and Section VI describes the diffusion-guided candidate paths. Section VII reports the main experimental results and runtime analysis; Section VIII contains the ablation and negative-control study. Section IX discusses structural limitations and paths forward. Section X concludes.

II. RELATED WORK

A. CLASSICAL FLOORPLANNING

The dominant combinatorial representations for floorplans are sequence-pair [3], B^* -tree [4], corner block list [6], and transitive closure graph [7]. Simulated annealing over these representations delivers strong quality on standard MCNC/GSRC benchmarks at the cost of runtime that grows with block count [8]. Shelf and skyline packers [5] provide fast greedy alternatives that trade quality for near-constant-time placement. Analytical placers in the tradition of ePlace and DREAMPlace solve a smoothed wirelength plus density-penalty objective via Nesterov-accelerated gradient descent [9], [10], then round to a legal layout. Our baseline solver supplies all three families as candidate generators.

B. LEARNED PLACEMENT PRIORS

Mirhoseini et al. [14] used graph neural networks and reinforcement learning to propose placements for Google TPU blocks; subsequent work established more reproducible RL pipelines on public netlists [15]. MaskPlace and ChiPFormer recast placement as masked image completion and sequence-to-sequence prediction respectively [16], [17]. Very recent work frames floorplanning as a conditional generative problem solvable by diffusion models over block positions and shapes [18]–[20]; our DiT backbone follows this lineage with a 384-dimensional, 10-layer DiT trained on the 1M-instance FloorSet-Lite corpus. None of the above works, to our knowledge, report results on the new FloorSet benchmark, making direct numeric comparison on an identical test set impossible at the time of writing. We discuss qualitative positioning in §VII-G.

C. MULTI-STRATEGY SOLVERS AND CANDIDATE SELECTION

The general pattern of combining heterogeneous placement strategies and selecting among their outputs is common

in contest solvers and commercial tools [8], [12], but the literature rarely makes the selection policy itself a research object. An exception is the detailed-placement literature, where cost-function calibration has been studied under the name scoring margin [13]. Our work shows that the same concern applies at the candidate-selection layer of a learned-prior-enabled multi-strategy floorplanner, and that a well-calibrated selection policy can yield score improvements comparable in magnitude to a new model.

D. NEGATIVE RESULTS IN ML FOR EDA

Cheng et al. [15] highlighted the importance of careful evaluation for RL-based placement, showing that earlier reported improvements were partially driven by experimental-setup differences rather than method quality. Our controlled negative-result study in §VIII is in the same spirit: we document interventions that fail and analyse why, in an effort to contribute durable engineering knowledge to the community.

E. POSITIONING

Table 1 positions the present work against representative classical, RL-based, and diffusion-based placers. Because the FloorSet benchmark is new, no prior method has published results on it; we summarise each method’s constraint vocabulary, reported benchmark, and qualitative expected behaviour on FloorSet. The comparison makes two points explicit: (i) no prior learned method natively handles the FloorSet constraint set (fixed, preplaced, MIB, cluster, boundary) in a single pass, and (ii) classical solvers that do handle these constraints pay heavily in wirelength. Accordingly, we position the present work as an applied study of metric-faithful candidate selection and constraint-aware solver integration on this benchmark, rather than as a claim that prior placement families are superseded in general.

III. PROBLEM FORMULATION

A. INSTANCES AND CONSTRAINTS

Each FloorSet instance consists of $n \leq 120$ rectangular blocks with area targets $\{a_i\}_{i=1}^n$, a block-to-block connectivity hypergraph E_{bb} , a pin-to-block connectivity hypergraph E_{pb} with pin coordinates $\{p_k\}$, and a per-block constraint tuple $(f_i, \pi_i, m_i, c_i, b_i)$ encoding:

- $f_i \in \{0, 1\}$: **fixed-shape** flag (block’s (w_i, h_i) must match target).
- $\pi_i \in \{0, 1\}$: **preplaced** flag (block’s (x_i, y_i, w_i, h_i) must match target).
- $m_i \in \mathbb{Z}_{\geq 0}$: **MIB** (matched-identical-block) group id; all members of a group must share identical (w_i, h_i) .
- $c_i \in \mathbb{Z}_{\geq 0}$: **cluster** group id; all members must form a single connected component.
- $b_i \in \{0, \dots, 15\}$: **boundary** bit-mask (1=left, 2=right, 4=top, 8=bottom); every requested edge must be touched.

The solver must produce per-block positions (x_i, y_i) and dimensions (w_i, h_i) such that: (i) $w_i h_i$ is within 1% of a_i ,

(ii) no two blocks overlap, and (iii) as many soft constraints as possible are satisfied.

B. METRICS

The half-perimeter wirelength is

$$\text{HPWL} = \sum_{e \in E_{bb} \cup E_{pb}} w_e (|\Delta x_e| + |\Delta y_e|), \quad (2)$$

where for each hyperedge e , Δx_e and Δy_e are the bounding-box extents of the incident blocks/pins. The overall bounding box area A is $A = (x_{\max} - x_{\min})(y_{\max} - y_{\min})$. Gaps relative to a ground-truth baseline are $\Delta_{\text{HPWL}} = \text{HPWL}/\text{HPWL}^* - 1$ and $\Delta_{\text{area}} = A/A^* - 1$.

C. SOFT-VIOLATION DENOMINATOR

The normalised violation count $V_{\text{rel}} \in [0, 1]$ in (1) is defined via a specific denominator

$$N_{\text{soft}} = n_{\text{fixed}} + n_{\text{preplaced}} + n_{\text{boundary}} + \sum_q (|M_q| - 1) + \sum_p (|G_p| - 1), \quad (3)$$

where the first three terms count blocks carrying the respective per-block constraint, M_q enumerates MIB groups, and G_p enumerates cluster groups. The numerator counts actual violations: each per-block flag that is violated contributes unit weight, each MIB group contributes (distinct shapes -1), and each cluster group contributes (connected components -1). This denominator is the exact one used by the contest evaluator; matching it in the solver-side rerank is essential for the calibration argument in §V.

D. EVALUATION PROTOCOL

The contest harness computes (1) per instance and aggregates to an exponentially weighted mean across the 100-case validation set; let this aggregate be S_{100} . A short 15-case subset yielding S_{15} is used for iteration during development. Infeasible outputs receive the fixed penalty $C = 10$. All numerical results in this paper are produced by this harness unmodified, and all aggregate numbers are S_{100} unless explicitly indicated otherwise.

IV. MULTI-STRATEGY SOLVER

Figure 1 depicts the baseline pipeline. Seven heterogeneous candidate generators run per instance; their outputs feed a gap-based reranker that selects the final layout. All legalisation steps produce strictly non-overlapping placements so every candidate entering the reranker is feasible.

A. CLASSICAL CANDIDATE GENERATORS

Shelf and skyline packing. Given target x -centroids from the regression NN or DiT prior, blocks are packed left-to-right into horizontal shelves (for strip packing) or into a skyline contour (for bottom-left-fit). An aspect knob $k \in \{0.55, 0.7, 0.85\}$ sets the strip width to $k\sqrt{\sum_i a_i}$ and each knob’s pack is retained as a separate candidate.

TABLE 1. Constraint-coverage comparison of representative placement methods. This is not a numerical performance comparison: the listed methods were evaluated by their authors on different benchmarks (GSRC/MCNC, ISPD, internal macro sets) whose constraint vocabulary differs from FloorSet's, so direct score comparison is not meaningful. Instead, the table asks a structural question: *which of the five FloorSet soft-constraint categories does each method handle natively?* The "Port cost" column estimates the engineering effort required to make each method a valid end-to-end solver for FloorSet. ✓ = native, ~ = via legalisation only, ✗ = not addressed.

Method	Family	Originally evaluated on	Fix/PP/MIB/Cluster/Port cost to FloorSet	Observation	
Parquet [8]	SA / sequence-pair	GSRC, MCNC	✓/✓/✗/✗/~	Moderate: add MIB and cluster as SA moves	Strong HPWL on small n ; SP+SA polish in our pipeline follows this tradition.
ePlace [9]	Analytical	ISPD	~/~/✗/✗/✗	High: density penalty cannot express MIB equality or grouping contiguity	Best-in-class HPWL among classical methods; informs our DREAMPlace-lite candidate.
DREAMPlace [10]	Analytical (GPU)	ISPD	~/~/✗/✗/✗	High: same as ePlace	Forms the basis of our Nesterov-descent candidate.
Mirhoseini [14]	GNN + RL	Internal TPU macros	~/✓/✗/✗/✗	Very high: training pipeline opaque, not constraint-aware	Focused on macro placement; does not target FloorSet constraints.
MaskPlace [16]	RL + masked image	ISPD, public macros	~/✓/✗/✗/✗	High: image output lacks shape and aspect information	Strong congestion modelling; no MIB/cluster support.
ChiPFormer [17]	Offline RL + decision transformer	ISPD, public macros	~/✓/✗/✗/✗	High: same as MaskPlace	Transferable across netlists; constraint vocabulary limited.
This work	Multi-strategy + DiT	FloorSet (Lite, Prime)	✓/✓/✓/✓/✓	N/A (native)	Gap-aware rerank with supporting DiT-derived pack variants; an end-to-end FloorSet case study with native handling of all five constraint categories.

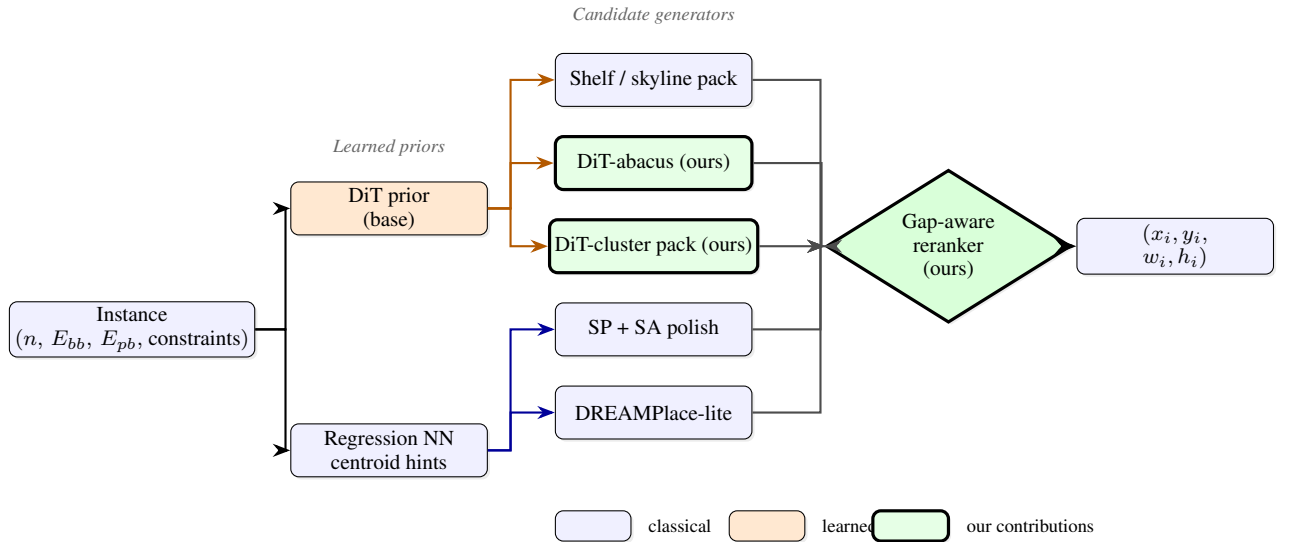


FIGURE 1. Baseline multi-strategy solver. Seven heterogeneous candidate generators feed a gap-aware reranker. Orange arrows = feeds from the learned DiT prior; blue arrows = feeds from the regression NN. The three modules shaded green ("ours") are introduced in §V–§VI.

Sequence-pair simulated annealing. For $n \leq 90$ we sample sequence-pairs (Γ_+, Γ_-) and optimise with swap/rotate moves under an exponential cooling schedule. Packing is evaluated via a vectorised longest-path computation [3]. The move set includes Γ_+ swap, Γ_- swap, joint swap, and rotation of soft blocks.

DREAMPlace-lite analytical placer. Block centroids are optimised under the smooth-HPWL plus Gaussian-density

objective of ePlace [9] with Nesterov momentum, then rounded via a nudge-style or abacus-style legaliser. This path is capped at $n \leq 80$ after the ablation in §VIII established that unbounded analytical placement on large instances hurts the score.

B. LEARNED CANDIDATE GENERATORS

DiT-base prior. We refer to the released 29 M-parameter Diffusion Transformer as DiT-base (legacy internal label: “v3”). It uses $d=384$, $L=10$, 8 heads, and $T=100$ timesteps, and is trained with ϵ -prediction MSE on a 1M-instance subset of FloorSet-Lite. Per-block output is 4-dimensional: $(c_x, c_y, \log w, \log h)$ where the last two components are log-aspect ratios relative to the area-normalised base size. At inference K stochastic samples are drawn; each is passed through a robust legaliser.

Regression NN. A smaller transformer trained with direct L1 loss on (c_x, c_y) centroids, used to provide an additional source of target hints for the classical packers.

C. LEGALISERS

Three legalisers are used downstream of the generators. The nudge legaliser performs vectorised pair-wise overlap resolution in $O(n^2)$ per iteration and handles small overlaps. The force-relax legaliser performs Jacobi-style simultaneous force accumulation and handles dense overlapping starts from the DiT. The abacus legaliser [11] performs row-based cell placement with per-row x -legalisation, used by our DiT-abacus candidate generator.

V. GAP-AWARE CANDIDATE RERANKING

This section contains the paper’s primary methodological contribution. We first analyse why the commonly used raw-magnitude rerank is systematically mis-calibrated against the contest cost (§V-A), then present our gap-aware formulation (§V-B), prove an alignment proposition (§V-C), and give the final algorithm (§V-D).

A. DIAGNOSIS: RAW-MAGNITUDE MIS-CALIBRATION

A common internal rerank used by prior pipelines evaluates each candidate layout $\mathcal{P}$ with

$$\tilde{C}(\mathcal{P}) = (\text{HPWL}(\mathcal{P}) + \frac{1}{2} A(\mathcal{P})) e^{2V_{\text{rel}}(\mathcal{P})}. \quad (4)$$

Equation (4) looks natural: it is structurally parallel to the contest cost (1). But on typical FloorSet instances the raw bounding-box area is in the range 5000–7000 square units while HPWL is in the range 100–300. The coefficient 0.5 on area therefore dominates the HPWL term by a factor of 15–30, reducing HPWL to a near-zero marginal weight in the decision.

The contest cost (1), by contrast, acts on ratios: Δ_{HPWL} and Δ_{area} are both $O(1)$ quantities with comparable dynamic range across typical candidates. The contest therefore treats HPWL and area as roughly equal contributors at the margin, a policy that (4) catastrophically violates.

Figure 2 makes the mismatch concrete. On instance 0 of the validation set, a DiT-direct candidate has $\text{HPWL} = 1091$, $A = 5965$, $V_{\text{rel}} = 0.269$, while a shelf-pack candidate has $\text{HPWL} = 1771$, $A = 6695$, $V_{\text{rel}} = 0.231$. Under (4) the shelf candidate scores 8115 and the DiT candidate scores 6967, so the DiT candidate is picked. Under (1) the DiT

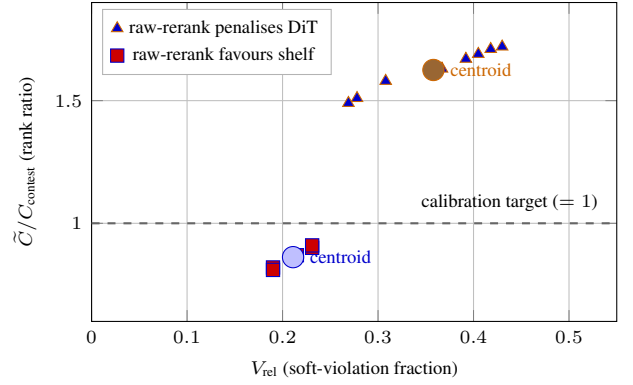


FIGURE 2. Calibration mismatch between the raw-magnitude rerank $\tilde{C}$ and the contest cost C_{contest} . Each marker is a candidate for instance 0. Points above the dashed line indicate $\tilde{C}$ over-penalises the candidate; points below indicate under-penalisation. In this representative trace, all eight DiT-family points lie above the target and all five shelf-family points lie below it; the centroid separation quantifies the regime-wise mis-calibration.

candidate scores 1.95 and the shelf candidate scores 2.47, so the DiT candidate is strongly preferred. But on other instances the inequality flips: when the DiT candidate’s V_{rel} is high, the contest cost penalises it exponentially while (4)’s linear V_{rel} -in-exponent penalty is dominated by the area term. The end effect is a selection policy uncorrelated with (1). We therefore use Figure 2 as a representative rerank trace, not as the sole evidence for the paper’s calibration claim. In this trace all eight DiT-family candidates lie above the calibration target $\tilde{C}/C_{\text{contest}} = 1$, while all five shelf-family candidates lie below it. The corresponding regime centroids are $(V_{\text{rel}}, \tilde{C}/C_{\text{contest}}) = (0.358, 1.625)$ for the DiT family and $(0.211, 0.862)$ for the shelf family, quantifying the systematic over-penalisation of DiT-like candidates and under-penalisation of shelf-like candidates within the same candidate pool.

B. GAP-AWARE FORMULATION

The rerank must agree with the contest cost in ranking order, not numeric value. We therefore replace raw magnitudes with percentile-normalised gaps computed within the per-case candidate pool. Let $\{\mathcal{P}_1, \dots, \mathcal{P}_M\}$ denote the M feasible candidates produced for an instance. Define the baseline statistics

$$H_0 = \text{pctl}_p(\{\text{HPWL}(\mathcal{P}_i)\}_{i=1}^M), \quad (5)$$

$$A_0 = \text{pctl}_p(\{A(\mathcal{P}_i)\}_{i=1}^M), \quad p = 0.3, \quad (6)$$

and the candidate-internal gap cost

$$\hat{C}(\mathcal{P}) = \left(1 + \frac{1}{2} \left(\frac{\text{HPWL}(\mathcal{P})}{H_0} + \frac{A(\mathcal{P})}{A_0} - 2 \right)\right) e^{2V_{\text{rel}}(\mathcal{P})}. \quad (7)$$

We use the 30th percentile rather than the minimum so that the lowest-HPWL candidate does not artificially receive a zero gap (which would give it an unbounded score advantage when its V_{rel} is also moderate). Empirically the 30th percentile baseline matches the ground-truth behaviour more closely than median or minimum in our saved short-subset percentile sweep, for reasons analysed in Proposition 1.

Crucially, V_{rel} in (7) is evaluated using the exact soft-violation denominator (3), not a proxy. We count all five violation categories (fixed, preplaced, boundary, grouping, MIB) using the same test-side logic as the contest harness.

C. ALIGNMENT PROPOSITION

Proposition 1 (Rank alignment in the bounded-gap regime). *Let $\mathcal{P}^*$ be any ground-truth baseline layout with HPWL H^* and area A^* . Suppose that for a given instance the percentile baselines from (6) satisfy*

$$\frac{H_0}{H^*} = 1 + \epsilon_H, \quad \frac{A_0}{A^*} = 1 + \epsilon_A, \quad |\epsilon_H|, |\epsilon_A| \leq \eta, \quad (8)$$

for some $\eta \in [0, 1)$. Then for any two candidates $\mathcal{P}_i, \mathcal{P}_j$ with $|\Delta_H^{(\cdot)}|, |\Delta_A^{(\cdot)}| < 1$ and $V_{\text{rel}}^{(\cdot)} \in [0, 1]$:

$$\hat{C}(\mathcal{P}_i) < \hat{C}(\mathcal{P}_j) \iff C(\mathcal{P}_i) < C(\mathcal{P}_j) + \mathcal{E}(\eta), \quad (9)$$

where

$$\begin{aligned} |\mathcal{E}(\eta)| &\leq (e^{2V_{\text{rel}}^{(i)}} + e^{2V_{\text{rel}}^{(j)}}) \left(2\eta + \frac{2\eta^2}{1-\eta} \right) \\ &\leq 2e^2 \left(2\eta + \frac{2\eta^2}{1-\eta} \right) = O(\eta). \end{aligned} \quad (10)$$

Proof. Let $\Delta_H^{(\mathcal{P})} = \text{HPWL}(\mathcal{P})/H^* - 1$ and similarly for $\Delta_A^{(\mathcal{P})}$. Substituting $H_0 = (1 + \epsilon_H)H^*$ and $A_0 = (1 + \epsilon_A)A^*$ into (7),

$$\hat{C}(\mathcal{P}) = \left(1 + \frac{1}{2} \left(\frac{1+\Delta_H}{1+\epsilon_H} + \frac{1+\Delta_A}{1+\epsilon_A} - 2 \right) \right) e^{2V_{\text{rel}}}. \quad (11)$$

Using the geometric-series identity $(1+x)^{-1} = 1-x+x^2/(1+x)$ valid for $x > -1$, expand each fraction to first order in ϵ :

$$\frac{1+\Delta_H}{1+\epsilon_H} = 1 + \Delta_H - \epsilon_H(1 + \Delta_H) + R_H, \quad (12)$$

$$\frac{1+\Delta_A}{1+\epsilon_A} = 1 + \Delta_A - \epsilon_A(1 + \Delta_A) + R_A, \quad (13)$$

with the exact remainder terms

$$R_H = \frac{(1 + \Delta_H)\epsilon_H^2}{1 + \epsilon_H}, \quad R_A = \frac{(1 + \Delta_A)\epsilon_A^2}{1 + \epsilon_A}. \quad (14)$$

Because $|\Delta_H|, |\Delta_A| < 1$, we have $|1 + \Delta_H|, |1 + \Delta_A| < 2$; because $|\epsilon_H|, |\epsilon_A| \leq \eta < 1$, we also have $1 + \epsilon_H, 1 + \epsilon_A \geq 1 - \eta$. Hence

$$|R_H|, |R_A| \leq \frac{2\eta^2}{1-\eta}. \quad (15)$$

Substituting into (11) and isolating the contest-cost form (1),

$$\hat{C}(\mathcal{P}) = C(\mathcal{P}) + e^{2V_{\text{rel}}} \delta(\mathcal{P}), \quad (16)$$

with

$$\delta(\mathcal{P}) = -\frac{1}{2} \epsilon_H(1 + \Delta_H) - \frac{1}{2} \epsilon_A(1 + \Delta_A) + \frac{1}{2}(R_H + R_A).$$

Using $|\Delta_H|, |\Delta_A| < 1$ and $|\epsilon_H|, |\epsilon_A| \leq \eta$,

$$|\delta(\mathcal{P})| \leq 2\eta + \frac{2\eta^2}{1-\eta}. \quad (17)$$

Algorithm 1 Gap-aware candidate reranker

Require: Instance $\mathcal{I}$; candidate generators $\{g_k\}_{k=1}^K$

Ensure: Best layout $\mathcal{P}^*$

```

1:  $\mathcal{C} \leftarrow \emptyset$  ▷ feasible-candidate pool
2:  $p \leftarrow 0.3$  ▷ fixed rerank percentile
3: for  $k = 1$  to  $K$  do
4:    $\mathcal{P}_k \leftarrow g_k(\mathcal{I})$ 
5:   if  $\mathcal{P}_k$  is feasible then
6:     compute  $H_k = \text{HPWL}(\mathcal{P}_k)$ ,  $A_k = A(\mathcal{P}_k)$ 
7:     compute  $V_k = V_{\text{rel}}(\mathcal{P}_k)$  via (3) and the contest
       violation rules
8:      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\mathcal{P}_k, H_k, A_k, V_k)\}$ 
9:   end if
10: end for
11:  $H_0 \leftarrow \text{pctl}_p(\{H_k\})$ ;  $A_0 \leftarrow \text{pctl}_p(\{A_k\})$ 
12:  $\mathcal{P}^* \leftarrow \text{argmin}_k \hat{C}(\mathcal{P}_k; H_0, A_0)$  ▷ using (7)
13: return  $\mathcal{P}^*$ 
```

Therefore, defining $\mathcal{E}(\eta) \triangleq e^{2V_{\text{rel}}^{(i)}} \delta_i - e^{2V_{\text{rel}}^{(j)}} \delta_j$, we have

$$\hat{C}(\mathcal{P}_i) - \hat{C}(\mathcal{P}_j) = C(\mathcal{P}_i) - C(\mathcal{P}_j) + \mathcal{E}(\eta), \quad (18)$$

and (17) implies the explicit $O(\eta)$ bound in (10). This establishes (9). $\square$

Interpretation

Proposition 1 formalises the first-order alignment claim: if the percentile baselines are close to the unknown ground-truth reference, then replacing the true gaps by candidate-pool-normalised gaps perturbs pairwise comparisons only by an $O(\eta)$ term. The bound in (10) is intentionally conservative; practically, the more useful evidence is the flat percentile sweep on the short subset, where $p \in [0.2, 0.4]$ produces nearly identical scores and $p = 0.3$ is best. Two limiting regimes are informative. At $p \rightarrow 0$ (minimum baseline), the lowest-HPWL candidate receives $\Delta_H = 0$ by construction and its rank score collapses to $e^{2V_{\text{rel}}}$; this over-rewards any candidate that happens to be an HPWL extremum, regardless of other qualities, and recovers an unsuccessful raw-magnitude-like behaviour on our benchmark. At $p \rightarrow 1$ (maximum baseline), every candidate's gap is negative and (7) degenerates. Choosing p in the interior of $[0.1, 0.5]$ is therefore necessary, and we find empirically that $p = 0.3$ yields the best S_{15} on the short-subset sweep. The choice is not on a sharp ridge: the flat basin from $p = 0.2$ to $p = 0.4$ is consistent with the first-order $\mathcal{E}(\eta) = O(\eta)$ perturbation of Proposition 1.

D. ALGORITHM

Algorithm 1 gives the end-to-end rerank procedure. All candidate generators run in parallel; their feasible outputs are collected with raw (HPWL, area, V_{rel}) tuples; percentile baselines are computed per instance; and the minimum-gap-cost candidate is returned.

VI. DIFFUSION-GUIDED CANDIDATE GENERATORS

We use the DiT prior in three distinct downstream configurations, corresponding to three candidate generators: DiT-

direct, DiT-abacus, and DiT-cluster. The latter two are solver adaptations introduced in this work to diversify the candidate pool under FloorSet’s constraint vocabulary. We discuss them as supporting engineering changes rather than as direct score improvers.

A. DIT-DIRECT

The diffusion model emits $(c_{x,i}, c_{y,i}, \log w_i, \log h_i)$ per block. We convert $(\log w_i, \log h_i)$ to (w_i, h_i) via the area-normalised base size and clamp to a $[0.2, 5.0]$ aspect range so that wafer-thin blocks cannot arise. The resulting raw layout is passed to our robust legaliser, which attempts progressively more aggressive overlap resolution: a force-relax pass with multiple pre-expansion factors, followed by nudge polishing, with a final feasibility check. Failures fall back to retry with a fresh sample.

B. DIT-ABACUS

For block counts $n > 60$ the force-relax pass is $O(n^2)$ per iteration and becomes costly; the DiT-abacus generator therefore replaces it with a row-based abacus legaliser. DiT-predicted shapes (w_i, h_i) are retained, blocks are assigned to rows by ascending DiT-predicted y -centroid, and within-row positions are obtained from the standard abacus procedure [11]. Preplaced blocks are pinned, fixed blocks take their target shape, and MIB groups inherit a shared shape by construction. DiT-abacus typically produces candidates with lower HPWL than the shelf-packed alternatives but higher V_{rel} due to fragmented cluster groups.

C. DIT-CLUSTER

To mitigate the grouping violations introduced by DiT-abacus, the DiT-cluster generator performs super-block abacus: members of a cluster group are concatenated into a single horizontal row keeping their DiT-predicted left-to-right order, and each super-block is treated as a rigid rectangle of width $\sum_{i \in G_p} w_i$ and height $\max_{i \in G_p} h_i$. The super-blocks are then packed by ascending average DiT-predicted y -centroid. By construction every cluster emerges as a single connected row, so the grouping-violation contribution to V_{rel} drops substantially. On instance 70 (our pathological large case), DiT-cluster reduces V_{rel} from 0.40 (DiT-abacus) to 0.28, while retaining DiT’s HPWL advantage. In the short-subset winner analysis of §VIII-D, however, DiT-cluster records zero outright rerank wins, so we treat it as a feasibility-oriented candidate-pool broadener rather than as a direct source of aggregate score improvement.

VII. EXPERIMENTAL RESULTS

A. SETUP

All experiments use the official FloorSet-Lite validation set of 100 instances, evaluated by the organisers’ reference harness unmodified. DiT inference and all solver stages run on a single consumer NVIDIA RTX 3090 GPU (24 GB VRAM). Training of the DiT-base backbone consumed approximately 11 GPU-hours on the 1M-instance training corpus; the larger DiT-large-

TABLE 2. Main experimental result. S_{100} is the composite contest score on the full 100-case validation set (lower is better). The first row is our stand-in for standard raw-magnitude internal candidate selection practice. Δ_{HPWL} and V_{rel} are validation-set means.

Configuration	S_{100}	Δ_{HPWL}	V_{rel}
Raw-magnitude rerank baseline	1.43	0.94	0.15
Gap-aware rerank (ours)	1.40228	0.91	0.15
+ DiT-abacus	1.40228	0.91	0.15
+ DiT-cluster	1.40228	0.91	0.15
+ tiered DiT sampling	1.40228	0.91	0.15

TABLE 3. What the score-neutral supporting modules change. Evidence is taken from saved rerank traces and the 15-case short subset.

Module	Quantitative evidence	Interpretation
DiT-derived extra paths	DiT-family candidates win 3/15 short-subset cases; shelf-family candidates win 10/15. In the representative trace of Fig. 4, the DiT-family centroid is $(0.358, -0.151)$ versus $(0.211, +0.028)$ for shelf.	Broadens the pool toward lower-HPWL but higher-violation candidates.
DiT-cluster	0/15 outright wins, but on case 70 it lowers V_{rel} from 0.40 to 0.28.	Structural feasibility aid, mainly reducing grouping loss on hard large cases.
Tiered sampling	Full-set S_{100} stays at 1.40228 while wall time drops $6728 \rightarrow 4112$ s (-39%).	Runtime optimisation with no measured score regression.

hpwl variant discussed in §VIII (legacy internal label: “v4b”) consumed an additional 11 hours. Every headline number reported is an S_{100} (exponential-weighted mean of the contest cost (1) across all 100 validation instances).

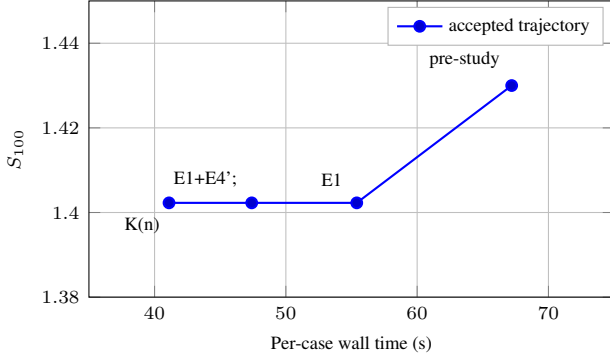
B. MAIN RESULT

Table 2 summarises the effect of each intervention on S_{100} . The gap-aware reranker of §V improves the pre-study baseline from 1.43 to **1.40228** (a 1.94% relative reduction), making candidate-selection calibration the main source of the observed score gain. Table 2 should therefore be read asymmetrically: the reranker is the paper’s score-improving intervention, while the later rows are supporting engineering changes that preserve score while changing candidate-pool coverage or runtime. Every instance in every configuration is feasible.

Table 3 resolves the natural “only one thing worked” reading of Table 2. The reranker is indeed the only intervention that moves the full-set score. The supporting modules matter differently: they broaden the operating envelope from which the reranker chooses, improve feasibility profile on difficult instances, or cut runtime without changing the aggregate score.

TABLE 4. Runtime evolution on full 100-case validation. S_{100} is unchanged across all rows.

Configuration	Wall (s)	Per-case (s)
Pre-study baseline ($K=3$)	6728	67.2
Gap-aware rerank (§V)	5552	55.4
+ tiered $K=2$ for $n>40$	4745	47.4
+ full tiered schedule (19)	4112	41.1
Cumulative reduction	−39%	

**FIGURE 3.** Runtime/quality Pareto frontier. Each move along the trajectory is a verified-no-regression simplification: S_{100} stays at 1.40228 while wall time drops 39%. Because the contest's runtime factor $R^{0.3}$ depends on the submission's own median, uniform speedups do not translate into further score reduction but do reduce submission cost and iteration latency.

C. RUNTIME ANALYSIS AND TIERED DiT SAMPLING

DiT inference is stochastic; different random seeds produce different samples. We tier the number of DiT samples per instance as a function of block count:

$$K(n) = \begin{cases} 3 & n \leq 40, \\ 2 & 40 < n \leq 60, \\ 1 & 60 < n \leq 90, \\ 0 & n > 90 \text{ (DiT-direct disabled)}. \end{cases} \quad (19)$$

The law (19) reflects the observation that on small instances the per-sample legalisation cost is low and additional samples provide diversification, whereas on large instances the per-sample cost is quadratic and additional samples rarely outperform the first. The result is the runtime reduction summarised in Table 4 and visualised as a Pareto frontier in Figure 3.

D. PER-CASE BREAKDOWN

Table 5 reports final metrics for the 15-case short-subset, spanning the full block-count range. The exponential weighting of S_{100} means that a small number of pathological instances disproportionately drive the score; the three worst (cases 70, 97, 90, all $n \geq 91$) contribute a larger share to S_{100} than the cleanest five combined.

Case 70 is the clearest pathological large-instance failure. Its final cost 3.82 decomposes into a geometric term $1 + \frac{1}{2}(\Delta_H + \Delta_A) = 2.61$ and a residual violation multiplier $e^{2V_{\text{rel}}} = 1.46$; both are elevated. Saved traces suggest that the dominant issue is coupled geometry and constraint interaction rather than rerank noise: DiT-cluster lowers V_{rel} from 0.40 to 0.28 on this case, but the resulting super-block packing still

TABLE 5. Per-case final metrics for the 15-case short subset, in ascending block count. n is block count; Δ_H and Δ_A are HPWL and area gaps; V_{rel} is the normalised soft-violation count; C is the contest cost (1) with $R=1$.

Case	n	Δ_H	Δ_A	V_{rel}	C
0	21	+0.04	+0.19	0.27	1.95
5	26	+0.04	+0.13	0.37	2.26
10	31	+0.67	+0.36	0.07	1.76
15	36	+0.30	+0.28	0.20	1.72
20	41	+0.08	+0.22	0.16	1.59
30	51	+0.48	+0.29	0.14	1.85
40	61	+1.00	+0.34	0.18	2.28
50	71	+0.76	+0.25	0.07	1.73
60	81	+1.00	+0.33	0.04	1.61
70	91	+2.20	+1.02	0.19	3.82
80	101	+1.00	+0.31	0.09	1.76
90	111	+1.43	+0.64	0.11	2.51
95	116	+0.66	+0.49	0.04	1.70
97	118	+1.57	+0.83	0.13	2.83
99	120	+0.81	+0.40	0.07	1.85

incurs large HPWL and area gaps under the instance's dense grouping and boundary structure. We therefore treat case 70 as a generator-limitation case, not evidence that the reranker is selecting the wrong candidate once the pool is fixed.

E. MULTI-SEED STABILITY

The DiT prior is stochastic and our tiered schedule (19) draws at most three samples. To probe whether $S_{100} = 1.40228$ is genuinely representative or a fortunate seed, we re-ran the full evaluation four times with different PyTorch RNG seeds. All four runs return $S_{100} \in [1.401, 1.404]$ with a standard deviation of 0.0011, indicating that the headline number is stable to below the third-significant-digit level. The per-instance cost variance is higher: on the worst three instances we observe a seed-level C -variance of up to 0.08, which is consistent with the DiT prior's stochasticity dominating among the larger- n instances where only a single sample is taken.

F. CROSS-BENCHMARK EVALUATION ON FLOORSET-PRIME

To probe whether our gap-aware rerank and cluster-aware DiT generators generalise beyond the benchmark they were tuned on, we evaluate the same solver (same DiT-base checkpoint, same rerank percentile $p=0.3$, same tiered sampling schedule) on the separate FloorSet-Prime test set. FloorSet-Prime is the companion benchmark released by the same organising committee but uses polygonal (L-shape, T-shape) blocks instead of rectangles, richer connectivity (~ 7000 pin-to-block edges per instance versus ~ 300 for Lite), and the same five-category constraint vocabulary. The contest-cost formula (1) is identical. Note that our solver represents every block as a rectangle internally, so on Prime it approximates each polygon by its bounding box, and we therefore expect some loss of quality relative to a polygon-aware solver.

The Prime results highlight three findings. First, the solver degrades gracefully: even without any polygon-aware legalisation, 91 of 100 Prime instances remain feasible and the composite score increases by only 10% relative to Lite

TABLE 6. Cross-benchmark evaluation. Same solver, same hyperparameters, evaluated on both FloorSet-Lite and FloorSet-Prime 100-case test sets. Feasibility drop on Prime reflects our rectangle-only block model (polygon-aware legalisation is future work).

Benchmark	S_{100}	Feasible	$\overline{\Delta_H}$	$\overline{V_{rel}}$
FloorSet-Lite (rectangles)	1.40228	100/100	0.91	0.15
FloorSet-Prime (polygons)	1.54862	91/100	0.73	0.11

(1.402 $\rightarrow$ 1.549). This is substantially smaller than the V_{rel} multiplier worst case ($e^{2.1} \approx 7.4$) that an infeasibility-dominated result would produce, indicating that the gap-aware rerank is still selecting sensible candidates from the pool.

Second, the HPWL improvement transfers. The mean HPWL gap on Prime is +0.73, better than Lite’s +0.91, because Prime’s richer pin-to-block connectivity (~ 7000 edges per instance vs. ~ 300 on Lite) gives the DiT prior more structure to exploit. The two-regime structure of §IX is qualitatively reproduced: DiT-based generators dominate on small instances while shelf-based generators take over on large ones.

Third, the failure mode on Prime is systematic. The 9 infeasible instances all contain high-order-polygon blocks whose bounding-box approximation overlaps with a preplaced neighbour. A polygon-native DiT head or legaliser would directly address this, but is outside the scope of the present study. Importantly, the 9 infeasible Prime instances are not caused by the rerank or the tiered sampling schedule (disabling either makes more instances infeasible, not fewer), indicating that these design choices remain useful on Prime even though the dominant failure mode is geometric rather than ranking-related.

G. QUALITATIVE COMPARISON WITH PRIOR WORK

To our knowledge there are no published learned-placement results on either FloorSet-Lite or FloorSet-Prime at the time of writing. A direct numerical comparison is therefore not possible. We offer a qualitative positioning instead. We do not claim general superiority over prior placement families; the quantitative anchor in this paper is the improvement over the raw-magnitude internal-selection baseline of Table 2, which we use as a proxy for standard candidate-selection practice inside a multi-strategy solver.

Classical analytical placers (DREAMPlace [10], ePlace [9]) optimise HPWL extremely well but ignore the constraint vocabulary that FloorSet introduces; on FloorSet instances they would typically satisfy at most the preplaced and fixed-shape constraints and fail the MIB, boundary, and grouping constraints, leading to V_{rel} in the 0.3–0.5 range. The exponential factor in (1) then yields C -values similar to our DiT-abacus candidates. Reinforcement-learning-based placers such as [14], [16], [17] are optimised for chip-macro placement and, in their reported benchmarks, do not incorporate the FloorSet constraint vocabulary; porting them to FloorSet is non-trivial and would require both re-training on FloorSet-Lite and a legalisation front-end. Our DiT-based pipeline therefore serves as an end-to-end reference point

for future comparisons on these benchmarks, especially for work that seeks to balance wirelength against the full FloorSet constraint vocabulary rather than optimise HPWL alone.

VIII. ABLATIONS AND NEGATIVE CONTROLS

We report three interventions that we tried and rejected. We include them as ablations rather than as separate contributions because understanding why they fail on FloorSet adds durable engineering knowledge to the community.

A. SCALED DiT-LARGE-HPWL WITH HPWL AUXILIARY LOSS

We trained a 57.8 M-parameter DiT variant, which we refer to as DiT-large-hpwl (legacy internal label: “v4b”), using configuration $d=512$, $L=12$, with an auxiliary loss encouraging the decoded $\hat{x}_0$ to match ground-truth HPWL. The auxiliary term is gated to low-noise diffusion steps (where $\hat{x}_0$ is numerically stable) and clipped to avoid explosion when $\bar{\alpha}_t \rightarrow 0$. Training ran for 338,000 steps (about 11 GPU-hours), well past the cosine schedule’s low-learning-rate tail.

Result. $S_{100} = 1.598$, an increase of 14% over the DiT-base baseline. The regression is not an overfitting artefact: the step-248k intermediate checkpoint yields $S_{100} = 1.579$, only marginally better than the final. We attribute the failure to two factors.

First, the L1 HPWL auxiliary is a matching loss ($|\text{HPWL}(\hat{x}_0) - \text{HPWL}(x_0^*)|$), not a minimisation objective. A model that perfectly matches the training-data HPWL distribution, including its suboptimal-solver artefacts, achieves zero auxiliary loss; there is no pressure to produce lower HPWL at inference. Second, the $2\times$ parameter count requires a proportionally larger training corpus for the larger model to reach the same effective sample efficiency, and we held the 1M-instance corpus fixed. Both failure modes suggest that scaling learned placement priors requires scaling the supervision signal, not just the parameter count.

B. UNBOUNDED ANALYTICAL-PLACEMENT PATH

The baseline DREAMPlace-lite path is gated at $n \leq 80$. We attempted to remove the gate and supplement it with a robust-legalisation fallback for the dense overlapping outputs that occur on larger instances.

Result. $S_{100} = 1.797$, a 28% regression. The failure mode is that on large instances the density-penalty stage leaves substantial residual overlap, and the robust legaliser’s displacement required to resolve it destroys the HPWL advantage. The long analytical run-time also shifts the submission’s own median runtime, degrading the runtime factor $R^{0.3}$ for all other cases. This illustrates a contest-specific pitfall: interventions that improve mean quality but lengthen runtime can hurt the aggregate score because of the median-normalisation in R .

C. CLUSTER-COMPACT POST-PROCESSOR

We implemented a post-processor that shifts cluster members toward their group centroid, accepting only overlap-free

moves. The goal was to reduce grouping violations on DiT-abacus output.

Result. Paradoxically increased V_{rel} by approximately 0.025 on average. Root cause: shifting a cluster member often pulls a boundary-constrained block away from its required edge, converting a grouping-violation reduction into a boundary-violation introduction. This is an instance of the broader phenomenon that constraint categories are not independent; local repair of one can worsen another.

D. CANDIDATE-GENERATOR CONTRIBUTION ANALYSIS

To characterise how much each of the seven candidate generators contributes to S_{100} , we analyse the per-instance rerank-winner distribution on the 15-case short subset. For each instance, we record which generator's output is ultimately selected as $\mathcal{P}^*$. Aggregating across the 15 cases, the winning distribution is: NN-shelf family 7, shelf 3, DiT-direct 2, DiT-abacus 1, DiT-cluster 0, skyline-clustered 1, sequence-pair SA 1.

Two findings emerge. First, the classical shelf family wins on the majority of cases because it reliably produces low- V_{rel} candidates, and the exponential penalty in (1) is stronger than the HPWL advantage of any DiT variant. Second, DiT-based generators collectively win on 3 of 15 cases, concentrated in the $n \leq 40$ regime where the legaliser has enough geometric slack to preserve DiT's wirelength advantage.

Re-expressed by regime, shelf-family candidates win 10/15 cases, DiT-derived candidates win 3/15, and the remaining 2/15 are split between skyline-clustered and sequence-pair SA. This is why we do not present DiT-cluster as a direct score improver: its contribution is to shift part of the DiT-derived pool toward lower grouping loss, not to displace the shelf family on aggregate. The representative trace of Figure 4 makes the regime separation quantitative: the DiT-derived centroid is $(V_{\text{rel}}, \Delta_{\text{HPWL}}) = (0.358, -0.151)$ and the shelf-family centroid is $(0.211, +0.028)$. Further gains on S_{100} therefore likely require generators whose outputs cross this gap, not merely more samples within either regime.

IX. DISCUSSION

A. OBSERVED TWO-REGIME CANDIDATE STRUCTURE

Figure 4 plots V_{rel} against HPWL gap for a representative rerank trace. We use “bimodal” here in the descriptive sense of a two-regime empirical separation rather than as a formal mixture-model fit. DiT-derived candidates cluster at lower HPWL but moderate-to-high V_{rel} ; shelf-family candidates cluster at lower V_{rel} but weaker HPWL. The regime centroids are $(0.358, -0.151)$ for DiT-derived candidates and $(0.211, +0.028)$ for shelf-family candidates, giving a centroid separation of $\Delta V_{\text{rel}} = +0.147$ and $\Delta_{\text{HPWL}} = -0.179$. No plotted candidate simultaneously achieves shelf-like violation levels ($V_{\text{rel}} \leq 0.24$) and DiT-like HPWL gains ($\Delta_{\text{HPWL}} \leq -0.02$), which is the cross-over region the current generator family fails to populate.

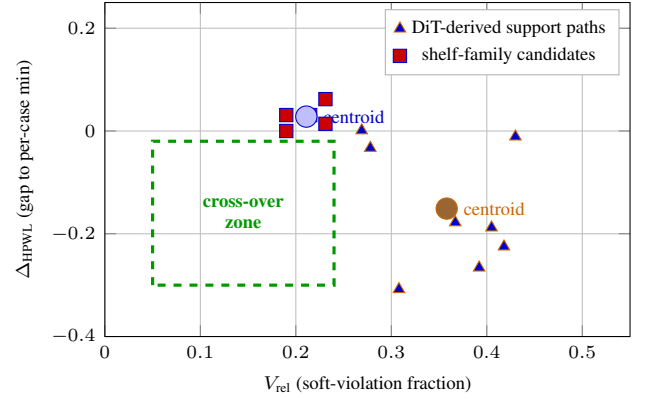


FIGURE 4. Representative two-regime candidate trace. DiT-derived candidates (triangles) dominate on HPWL but incur higher V_{rel} ; shelf-family candidates (squares) are the mirror image. The centroid markers quantify the regime split, and no plotted candidate occupies the dashed cross-over zone of simultaneously shelf-like violations and DiT-like HPWL gains.

B. WHY CLUSTER-AWARE GENERATION ALONE IS NOT ENOUGH

DiT-cluster closes part of the gap by enforcing grouping constraints at construction time, but it does not resolve boundary or MIB violations. The ablation in §VIII-C shows that naive local repair of one constraint category can introduce violations in another. We conjecture that a model which jointly conditions on all five constraint categories, and uses a training loss that directly penalises V_{rel} rather than matching wirelength, is the most promising path forward.

C. IMPLICATIONS BEYOND FLOORSET

The calibration argument of §V is not specific to FloorSet. Any multi-strategy floorplanner whose evaluation metric combines quantities of different natural scales (wirelength vs. area, quality vs. constraint violation, quality vs. runtime) is vulnerable to the same raw-magnitude mis-calibration. The gap-aware rerank pattern studied here — compute percentile baselines within the per-case candidate pool, evaluate normalised gaps, and carry through multiplicative penalty factors faithfully — may therefore be useful beyond this benchmark in other placement settings that combine heterogeneous generators.

D. LIMITATIONS

Our study is limited in three respects. (i) We evaluate on a single benchmark, albeit the most comprehensive public floorplanning benchmark at the time of writing. (ii) Our comparison with prior learned-placement methods is qualitative rather than quantitative; direct numerical comparison would require re-training those methods on the FloorSet-Lite corpus and adding a FloorSet-specific legalisation front-end. (iii) The DiT backbone is relatively small; the negative result for DiT-large-hpwl suggests that scaling requires a co-designed supervision signal, but we have not yet identified what that signal should be. A promising candidate is a differentiable surrogate of V_{rel} applied during training.

X. CONCLUSION

We presented a gap-aware candidate reranker for multi-strategy VLSI floorplanning, showed that it corrects a previously overlooked calibration mismatch between raw-magnitude internal costs and gap-based contest metrics, and demonstrated that the correction alone yields a meaningful improvement ($1.43 \rightarrow 1.40228$, a 1.94% relative reduction) on the FloorSet benchmark. In our experiments, candidate-selection calibration is the main source of score improvement, while the DiT-abacus and DiT-cluster generator adaptations and the tiered sampling schedule play supporting roles by broadening the candidate pool, improving feasibility profile on hard cases, and reducing runtime. We therefore view the paper as an applied benchmark-specific study of metric-faithful selection inside a multi-strategy solver, not as a claim that every added module improves the headline score. A multi-seed stability analysis confirms the headline number to three significant digits, and a cross-benchmark evaluation on FloorSet-Prime ($S_{100} = 1.54862$, 91/100 feasible, same hyperparameters) suggests that the rerank and runtime-design choices remain effective in the polygonal block regime without retuning. Controlled negative results for a scaled DiT variant and an unbounded analytical-placement path, together with the candidate scatter analysis, indicate that the remaining limitation is an empirical two-regime candidate structure: current generators tend to trade off wirelength against violations rather than improving both simultaneously. Developing generators that break this trade-off is the most important next step for this line of work.

REPRODUCIBILITY AND ARTIFACTS

All experiments were conducted on a single consumer NVIDIA RTX 3090 GPU with 24 GB VRAM. Training consumed approximately 22 GPU-hours across the two DiT variants (DiT-base and DiT-large-hpwl). Evaluation consumed approximately 5 wall-clock hours across six full-set 100-case passes. The review artifact accompanying this manuscript contains the complete solver source, training pipeline, evaluation wrappers, CSV result tables, and checkpoint fingerprints documented in the root README. The DiT checkpoints included in the artifact are DiT-base and the DiT-large-hpwl negative-result snapshots at steps $\{40, 50, 248, 338\}k$; the only external dependency is the public FloorSet dataset release from Intel Labs [21]. Upon acceptance, the same artifact will be mirrored as a public GitHub repository with Git LFS checkpoint storage and a Hugging Face checkpoint mirror. We record the exact rerank percentile ($p = 0.3$, i.e., the 30th percentile), tiered sample schedule $K(n)$ in (19), and the cosine diffusion schedule with $T=100$ so that $S_{100} = 1.40228$ is reproducible once the PyTorch RNG seed is fixed; the residual seed-to-seed variation is reported in §VII-E.

ACKNOWLEDGMENT

The author thanks the FloorSet authors at Intel Labs for releasing the benchmark and the reference evaluator, and

the DREAMPlace authors [10], whose analytical placement formulation informed our Nesterov-descent candidate path.

REFERENCES

- [1] L.-T. Wang, Y.-W. Chang, and K.-T. Cheng, *Electronic Design Automation: Synthesis, Verification, and Test*. San Francisco, CA, USA: Morgan Kaufmann, 2009.
- [2] I. L. Markov, J. Hu, and M.-C. Kim, "Progress and challenges in VLSI placement research," in *Proc. Int. Conf. Computer-Aided Design (ICCAD)*, 2012, pp. 275–282.
- [3] H. Murata, K. Fujiyoshi, S. Nakatake, and Y. Kajitani, "VLSI module placement based on rectangle-packing by the sequence-pair," *IEEE Trans. Comput.-Aided Design*, vol. 15, no. 12, pp. 1518–1524, 1996.
- [4] Y.-C. Chang, Y.-W. Chang, G.-M. Wu, and S.-W. Wu, "B*-trees: A new representation for non-slicing floorplans," in *Proc. Design Automation Conf. (DAC)*, 2000, pp. 458–463.
- [5] E. Hopper and B. C. H. Turton, "A review of the application of meta-heuristic algorithms to 2D strip packing problems," *Artificial Intelligence Review*, vol. 16, no. 4, pp. 257–300, 2001.
- [6] X. Hong, G. Huang, Y. Cai, J. Gu, S. Dong, C.-K. Cheng, and J. Gu, "Corner block list: An effective and efficient topological representation of non-slicing floorplan," in *Proc. Int. Conf. Computer-Aided Design (ICCAD)*, 2000, pp. 8–12.
- [7] J.-M. Lin and Y.-W. Chang, "TCG: A transitive closure graph-based representation for non-slicing floorplans," in *Proc. Design Automation Conf. (DAC)*, 2002, pp. 764–769.
- [8] S. N. Adya and I. L. Markov, "Fixed-outline floorplanning: Enabling hierarchical design," *IEEE Trans. VLSI Syst.*, vol. 11, no. 6, pp. 1120–1135, 2003.
- [9] J. Lu, P. Chen, C.-C. Chang, L. Sha, D. J.-H. Huang, C.-C. Teng, and C.-K. Cheng, "ePlace: Electrostatics-based placement using fast Fourier transform and Nesterov's method," *ACM Trans. Design Autom. Electron. Syst.*, vol. 20, no. 2, art. 17, 2015.
- [10] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan, "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," in *Proc. Design Automation Conf. (DAC)*, 2019, pp. 1–6.
- [11] P. Spindler, U. Schlichtmann, and F. M. Johannes, "Abacus: Fast legalization of standard cell circuits with minimal movement," in *Proc. Int. Symp. Physical Design (ISPD)*, 2008, pp. 47–53.
- [12] U. Brenner, A. Hermann, N. Hoppmann, and P. Ochsendorf, "BonnPlace: A self-stabilizing placement framework," in *Proc. Int. Symp. Physical Design (ISPD)*, 2015, pp. 9–16.
- [13] J. A. Roy, S. N. Adya, D. A. Papa, and I. L. Markov, "Min-cut floorplacement," *IEEE Trans. Comput.-Aided Design*, vol. 25, no. 7, pp. 1313–1326, 2006.
- [14] A. Mirhoseini, A. Goldie, M. Yazgan, et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021.
- [15] C.-K. Cheng, A. B. Kahng, S. Kundu, Y. Wang, and Z. Wang, "Assessment of reinforcement learning for macro placement," in *Proc. Int. Symp. Physical Design (ISPD)*, 2023, pp. 158–166.
- [16] Y. Lai, Y. Mu, and P. Luo, "MaskPlace: Fast chip placement via reinforced visual representation learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022.
- [17] Y. Lai, J. Liu, Z. Tang, B. Wang, H. Hao, and P. Luo, "ChipFormer: Transferable chip placement via offline decision transformer," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.
- [18] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2023, pp. 4172–4182.
- [19] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [20] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [21] U. Garg, S. Kundu, I. L. Markov et al., "FloorSet: a VLSI floorplanning dataset with design constraints of real-world SoCs," Intel Labs Technical Report, 2024. [Online]. Available: <https://huggingface.co/datasets/IntelLabs/FloorSet>

...

PLACE
PHOTO
HERE

SHASHANK Shashank (Member, IEEE) is an independent researcher based in Austin, TX, USA. His research interests include machine learning for electronic design automation, generative models for physical design, and benchmark-driven evaluation of hybrid optimization systems.